Quest 1: The Coneservation

Lord of the Wings · topping up the pine-cone defences of Quackshire

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. Every number in the examples and traces was checked by running the code in this repo (`node partN.ts --sample`, plus small instrumented copies to print intermediate values).

In one sentence: each outpost has some pine cones; find the smallest valid formation that can use all of them, and add up how many cones are missing.
Input: one comma-separated list of numbers (cones per outpost).
Output: the total number of new cones to collect.
What changes between parts: only the shape of a valid formation.

1. The story

Before leaving Quackshire, the narrator and Frodoff check the village defences. The outposts protect the village with **formations of enchanted elven pine cones** laid out on a grid. Many cones have lost their magic, so each outpost has fewer than it needs. The outposts form **three defensive rings**, and each ring uses a different formation shape.

2. Understanding the question

Three rules apply in every part:

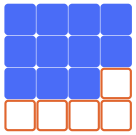
1. Each outpost builds **exactly one** formation.
2. It must use **all** of its cones. It can't leave any out or throw any away.
3. It can only **add** new cones.

So an outpost with `x` cones has to grow to the **next valid formation size**: the smallest one that holds at least `x` cones. The number of cones to collect is `size - x`, and the answer is the sum of this over all outposts.

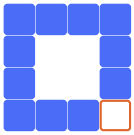
> **Think of it as:** a list of allowed sizes, such as 1, 4, 9, 16, 25, …. For each number, find the first allowed size that is at least that number, and measure the gap.

For example, 10 cones is one too many for a 3×3 square (9), so the outpost can't use 3×3. It has to go up to 4×4 (16) and collect 16 − 10 = **6** cones.

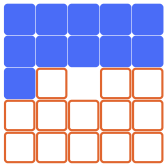
The three formation shapes



Ring 1: filled square
11 cones → 4×4, collect 5



Ring 2: square frame
11 cones → 4×4, collect 1



Ring 3: filled square, centre empty
11 cones → 5×5, collect 13

Blue cells are cones the outpost already has; orange outlines are cones it must collect.

Ring	Valid sides n	Cones needed	Valid sizes
1 (Part I)	1, 2, 3, 4, …	n^2	1, 4, 9, 16, 25, 36, …
2 (Part II)	3, 4, 5, 6, …	$4(n - 1)$	8, 12, 16, 20, 24, 28, …
3 (Part III)	3, 5, 7, 9, …	$n^2 - 1$	8, 24, 48, 80, 120, …

3. Reading the input (same in all three parts)

```ts
import { readFileSync } from "node:fs";

const isSample = process.argv.includes("--sample");
const file = isSample ? "sample1.txt" : "input1.txt";
const INPUTS: number[] = readFileSync(file, "utf8")
  .trim()
  .split(/[\s,]+/)
  .map(Number);
```

q1/part1.ts, lines 1–8

Lines	What happens
3–4	With `--sample` it reads `sample1.txt`; otherwise it reads `input1.txt` (part 2 and part 3 read `input2.txt` and `input3.txt`). All three parts share the same sample.
5–6	Reads the file as text and trims the trailing newline.
7	Splits on any run of commas or whitespace (`/[\s,]+/`), so `3,11, 12` and a list spread over several lines both work.
8	Turns each piece into a number: `"3,11,12"` becomes `[3, 11, 12]`.

4. Part I: filled squares

The question

A valid formation is a completely filled n×n square, so it holds `n²` cones. Any side n ≥ 1 is allowed:

```
1 x 1     2 x 2     3 x 3
#         ##        ###
          ##        ###
                    ###
```

Worked examples

Cones x	Smallest n with n² ≥ x	n²	Collect
3	2	4	1
11	4	16	5
12	4	16	4
16	4	16	0
24	5	25	1
25	5	25	0
Sample total			**11**

Extra examples

Cones x	Reasoning	Collect
1	already a 1×1 square	0
10	9 < 10 ≤ 16 → 4×4	6
50	49 < 50 ≤ 64 → 8×8	14
100	exactly 10×10	0

Step-by-step solution

1. Read the file and split it into a list of numbers `x` (one per outpost).
2. For one outpost, start with the smallest square: side `n = 1`.
3. If `n × n` is still smaller than `x`, the square is too small, so try `n + 1`. Repeat.
4. When `n × n ≥ x`, you have found the smallest square that fits. The cones to collect are `n × n − x`.
5. Do steps 2–4 for every outpost and add up the results. That total is the answer.

My solution

```
10  const getAnswer = (num: number) => {
11    let n = 1;
12
13    while (n * n < num) n++;
14
15    return n * n - num;
16  };
17
18  let sum = 0;
19
20  INPUTS.forEach((n) => {
21    sum += getAnswer(n);
22  });
23
24  console.log(sum);
```

q1/part1.ts, lines 10–24

Lines	What happens
10–11	`getAnswer` handles one outpost. It starts at the smallest square, side `n = 1`.
13	While the square is still too small (`n * n < num`), it tries the next side. When the loop stops, `n` is the smallest side that fits.
15	It returns how many cones are missing: `n² - num`.
18–22	It runs `getAnswer` on every outpost and adds the results into `sum`.
24	It prints the total.

Trace: `getAnswer(11)`

n	n * n	n * n < 11?	Action
1	1	yes	n++
2	4	yes	n++
3	9	yes	n++
4	16	no	stop → return 16 – 11 = **5**

5. Part II: square frames

The question

The second ring builds only the **border** of a square. The frame must surround at least one empty cell, or the magic fails, so 1×1 and 2×2 aren't allowed and the smallest frame is 3×3:

```
3 x 3     4 x 4     5 x 5
###       ####      #####
#.#       #..#      #...#
###       #..#      #...#
          ####      #...#
                    #####
```

Where 4(n – 1) comes from

Count the border of an n×n square: the top row has n cells and the bottom row has n cells. Each side column then adds n – 2 more, because its two corners are already counted. Total: `n + n + 2(n – 2) = 4n – 4 = 4(n – 1)`. For example, a 5×5 frame has 5 + 5 + 3 + 3 = 16 = 4 × 4 cones.

Worked examples

Cones x	Smallest n ≥ 3 with 4(n–1) ≥ x	4(n–1)	Collect
3	3	8	5
11	4	12	1
12	4	12	0
16	5	16	0
24	7	24	0
25	8	28	3
Sample total			**9**

Extra examples

Cones x	Reasoning	Collect
1	even 1 cone needs the smallest frame, 3×3 = 8	7
13	frame sizes go up in steps of 4: 12 < 13 ≤ 16 → 5×5	3
50	48 < 50 ≤ 52 → 14×14	2
100	4 × 25 = 100 → 26×26	0

Frame sizes go up by 4 each time, so an outpost never needs more than 3 extra cones, except when it has fewer than 8.

Step-by-step solution

1. Read the numbers, same as Part I.
2. For one outpost, start with the smallest valid frame: `n = 3`. Smaller frames have no empty cell inside.
3. Work out the frame size `4 × (n − 1)`. If it is smaller than `x`, try `n + 1`. Repeat.
4. When `4 × (n − 1) ≥ x`, the cones to collect are `4 × (n − 1) − x`.
5. Add this up over all outposts.

My solution

```
10  const getAnswer = (num: number) => {
11    let n = 3;
12
13    while (4 * (n - 1) < num) n++;
14
15    return 4 * (n - 1) - num;
16  };
```

q1/part2.ts, lines 10–16 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
11	Starts at `n = 3`, because the smallest valid frame is 3×3.
13	Grows the side while the frame is too small: `4 * (n - 1) < num`.
15	Returns the gap `4(n - 1) - num`.
rest	Same as Part I: sum over all outposts and print.

Trace: `getAnswer(25)`

n	4(n − 1)	< 25?
3	8	yes
4	12	yes
5	16	yes
6	20	yes
7	24	yes
8	28	no → return 28 − 25 = **3**

6. Part III: filled squares with an empty centre

The question

The third ring builds a filled square with **exactly one empty cell in the very centre**. A square with an even side has no single centre cell: the middle of a 4×4 falls between four cells. So only **odd** sides are allowed:

```
3 x 3     5 x 5     7 x 7
###       #####     #######
#.#       #####     #######
###       ##.##     #######
          #####     ###.###
          #####     #######
                    #######
                    #######
```

An n×n formation holds `n² − 1` cones (the full square minus the centre).

Worked examples

Cones x	Smallest odd n ≥ 3 with n² − 1 ≥ x	n² − 1	Collect
3	3	8	5
11	5	24	13
12	5	24	12
16	5	24	8
24	5	24	0
25	7	48	23
Sample total			**61**

Extra examples

Cones x	Reasoning	Collect
48	exactly 7×7 − 1	0
49	one too many for 7×7, so it has to jump to 9×9 = 80	31
100	80 < 100 ≤ 120 → 11×11	20

The gaps grow quickly because valid sizes jump by about 4n each step (8 → 24 → 48 → 80 …).

Step-by-step solution

1. Read the numbers, same as Part I.
2. For one outpost, start with `n = 3`, the smallest odd side.
3. Work out the formation size `n × n − 1`. If it is smaller than `x`, try `n + 2`, skipping even sides. Repeat.
4. When `n × n − 1 ≥ x`, the cones to collect are `n × n − 1 − x`.
5. Add this up over all outposts.

My solution

```
10  const getAnswer = (num: number) => {
11    let n = 3;
12
13    while (n * n - 1 < num) n += 2;
14
15    return n * n - 1 - num;
16  };
```

q1/part3.ts, lines 10–16 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
11	Starts at `n = 3`, the smallest odd side that can have an empty centre.
13	Grows the side by **2** (`n += 2`), so `n` stays odd: 3, 5, 7, ….
15	Returns `n² − 1 − num`.

Trace: `getAnswer(25)`

n	n² – 1	< 25?
3	8	yes
5	24	yes
7	48	no → return 48 – 25 = **23**

7. Summary

Part	Start n	Step	Capacity	Sample
I	1	+1	n^2	11
II	3	+1	$4(n-1)$	9
III	3	+2	n^2-1	61

> All three parts use the same pattern: **start at the smallest valid side, grow until the formation is big enough, and return the gap**. The loop runs about √x times, which is instant for these inputs. If you wanted a formula instead of a loop, Part I's side is `Math.ceil(Math.sqrt(x))`.